IqraSec Academy | WAPT Phase 1

Task 4: Web Vulnerabilities – SQL Injection

Intern name	Maryam Batool
Report title	Task 4: Common Web Vulnerabilities — SQL Injection
Rooms / Labs covered	PortSwigger Web Security Academy — SQL injection vulnerability in WHERE clause a
Date completed	1 November 2026

1. Lab Name

SQL injection vulnerability in WHERE clause allowing retrieval of hidden data (PortSwigger Web Security Academy).

2. Objective

To identify and exploit a SQL injection vulnerability in a product category filter, and use it to bypass the intended query logic and reveal products that should have been hidden from the user.

3. Vulnerability / Concept

- The application builds a SQL query by directly concatenating the category URL parameter into a WHERE clause, without sanitising or parameterising it.
- An attacker can break out of the intended string literal and inject their own SQL logic, changing what the query returns.
- The classic payload ' OR 1=1-- turns the WHERE condition into something that is always true for every row, so the query returns every product in the table regardless of category — including any that were meant to be hidden or unreleased.

4. Testing Approach

- Opened the shop and selected a category filter (e.g. Accessories) to see the normal, unmodified query behaviour and confirm the lab started in a "Not solved" state.
- Edited the category parameter in the URL from category=Accessories to category=Accessories' OR 1=1--.
- The trailing -- comments out the rest of the original SQL query (the part that would normally restrict results to the released/visible category), so only the injected condition is evaluated.
- Submitted the modified URL and observed that the page returned products from every category instead of just Accessories, confirming the injection worked.

5. Evidence / Screenshots

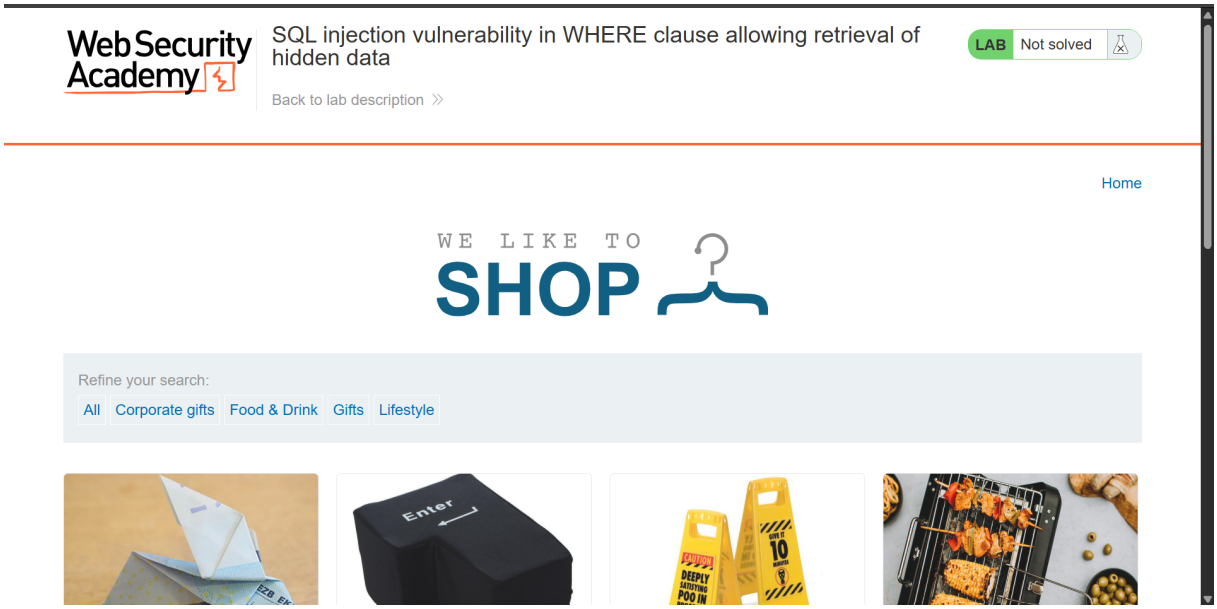


Figure 1: *Lab in its initial state — status shown as Not solved.*



Figure 2: *Injected URL category=Accessories' OR 1=1-- in the address bar; the lab now shows Solved and returns results across categories.*

6. Impact

An attacker exploiting this flaw can read data they are not authorised to see — in this lab, products outside the selected category, but the same technique against a real application could expose other users' data, authentication credentials, or any other information stored in the database. Depending on the database permissions, SQL injection can sometimes also be used to modify or delete data, or to escalate into the underlying server.

7. Remediation

- Use parameterised queries / prepared statements so user input is always treated as data, never as part of the SQL command itself.

- Apply least-privilege database accounts so even a successful injection has minimal access.
- Validate and allow-list expected input (e.g. the category parameter should only ever match a known, fixed set of category names).

8. What I Learned

- Even a simple, unvalidated filter parameter can be a serious vulnerability if it is concatenated directly into a SQL query.
- The ' OR 1=1-- pattern is a quick way to test whether a parameter is vulnerable, because it turns the WHERE clause into something universally true.
- Comment sequences like -- are just as important as the injected condition itself, since they neutralise the rest of the original query.

Declaration: I confirm that this report documents my own work and that all screenshots were taken by me during the tasks described.

Name: Maryam Batool Date: 1 November 2026